

**Some of the python expressions are:****Generator expression:****Syntax:** ( compute(var) for var in iterable )

```
>>> x = (i for i in 'abc') #tuple comprehension
>>> x
<generator object <genexpr> at 0x033EEC30>

>>> print(x)
<generator object <genexpr> at 0x033EEC30>
```

You might expect this to print as ('a', 'b', 'c') but it prints as <generator object <genexpr> at 0x02AAD710> The result of a tuple comprehension is not a tuple: it is actually a generator. The only thing that you need to know now about a generator now is that you can iterate over it, but ONLY ONCE.

**Conditional expression:****Syntax:** true\_value if Condition else false\_value

```
>>> x = "1" if True else "2"

>>> x

'1'
```

**Statements:**

A statement is an instruction that the Python interpreter can execute. We have normally two basic statements, the assignment statement and the print statement. Some other kinds of statements that are if statements, while statements, and for statements generally called as control flows.

Examples:

An assignment statement creates new variables and gives them values:

```
>>> x=10
```

```
>>> college="mrcet"
```

An print statement is something which is an input from the user, to be printed / displayed on to the screen (or ) monitor.

```
>>> print("mrcet colege")
```

```
mrcet college
```

### **Precedence of Operators:**

Operator precedence affects how an expression is evaluated.

For example,  $x = 7 + 3 * 2$ ; here, x is assigned 13, not 20 because operator \* has higher precedence than +, so it first multiplies  $3*2$  and then adds into 7.

#### **Example 1:**

```
>>> 3+4*2
```

```
11
```

Multiplication gets evaluated before the addition operation

```
>>> (10+10)*2
```

```
40
```

Parentheses () overriding the precedence of the arithmetic operators

#### **Example 2:**

```
a = 20
```

```
b = 10
```

```
c = 15
```

```
d = 5
```

```
e = 0
```

```
e = (a + b) * c / d    #( 30 * 15 ) / 5  
print("Value of (a + b) * c / d is ", e)
```

```
e = ((a + b) * c) / d    # (30 * 15 ) / 5  
print("Value of ((a + b) * c) / d is ", e)
```

```
e = (a + b) * (c / d);    # (30) * (15/5)
```

```
print("Value of (a + b) * (c / d) is ", e)
```

```
e = a + (b * c) / d;    # 20 + (150/5)
print("Value of a + (b * c) / d is ", e)
```

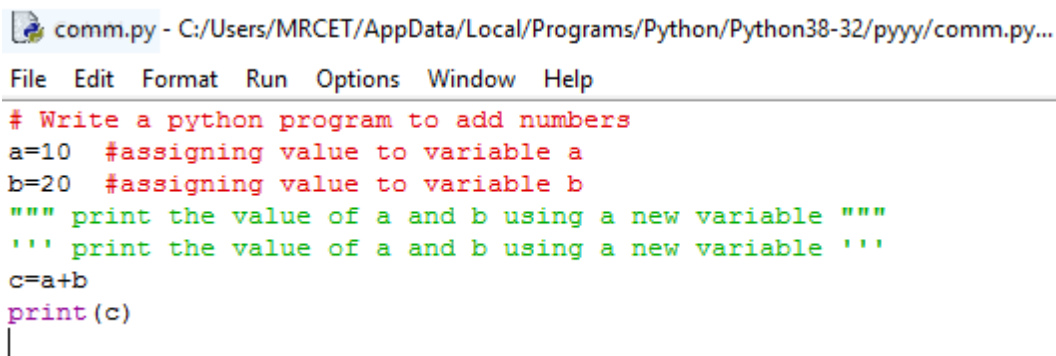
**Output:**

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/opprec.py
Value of (a + b) * c / d is 90.0
Value of ((a + b) * c) / d is 90.0
Value of (a + b) * (c / d) is 90.0
Value of a + (b * c) / d is 50.0
```

**Comments:**

**Single-line comments** begins with a hash(#) symbol and is useful in mentioning that the whole line should be considered as a comment until the end of line.

**A Multi line comment** is useful when we need to comment on many lines. In python, triple double quote(“ “ “) and single quote(‘ ‘ ‘)are used for multi-line commenting.

**Example:**

```
comm.py - C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/comm.py...
File Edit Format Run Options Window Help
# Write a python program to add numbers
a=10 #assigning value to variable a
b=20 #assigning value to variable b
""" print the value of a and b using a new variable """
''' print the value of a and b using a new variable '''
c=a+b
print(c)
|
```

**Output:**

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/comm.py
```

```
30
```

**Modules:**

**Modules:** Python module can be defined as a python program file which contains a python code including python functions, class, or variables. In other words, we can say that our python code file saved with the extension (.py) is treated as the module. We may have a runnable code inside the python module. A module in Python provides us the flexibility to organize the code in a logical way. To use the functionality of one module into another, we must have to **import** the specific module.

Syntax:

```
import <module-name>
```

Every module has its own functions, those can be accessed with . (dot)

**Note:** In python we have help ()

Enter the name of any module, keyword, or topic to get help on writing Python programs and using Python modules. To quit this help utility and return to the interpreter, just type "quit".

Some of the modules like os, date, and calendar so on.....

```
>>> import sys
```

```
>>> print(sys.version)
```

```
3.8.0 (tags/v3.8.0:fa919fd, Oct 14 2019, 19:21:23) [MSC v.1916 32 bit (Intel)]
```

```
>>> print(sys.version_info)
```

```
sys.version_info(major=3, minor=8, micro=0, releaselevel='final', serial=0)
```

```
>>> print(calendar.month(2021,5))
```

```
    May 2021
Mo Tu We Th Fr Sa Su
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
```

```
>>> print(calendar.isleap(2020))
```

```
True
```

```
>>> print(calendar.isleap(2017))
```

```
False
```